

DevOps = Development (Dev) + Operations (Ops)

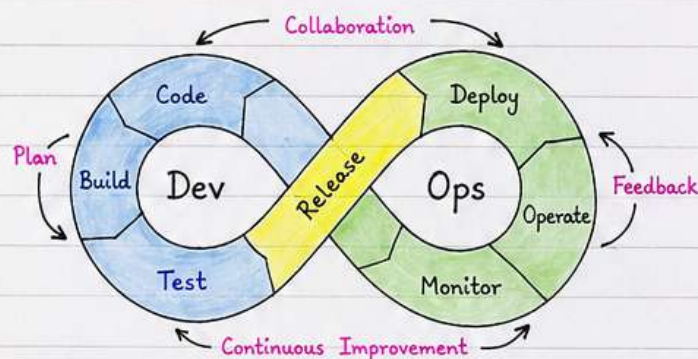
DevOps is a culture and set of practices that brings Development and Operations teams together to deliver software faster, more reliably and with high quality.

1. What is DevOps?

- DevOps is not a tool, it is a Mindset and Culture.
- It aims to shorten the SDLC and provide continuous delivery with high software quality.
- It breaks the silos between Development and Operations teams.
- It uses automation, monitoring and collaboration to improve the software delivery process.

Why DevOps?

- ✓ Faster Delivery
- ✓ Higher Quality
- ✓ Lower Failure Rate
- ✓ Improved Collaboration
- ✓ Customer Satisfaction



2. Goal of DevOps

1. Continuous Delivery / Deployment
2. Automation of processes
3. Continuous Integration
4. Continuous Monitoring
5. Rapid feedback and improvement

3. DevOps Principles

- ★ Customer-Centric Approach
- ★ Collaboration & Communication
- ★ Automation Everything
- ★ Continuous Integration & Delivery
- ★ Monitor Everything
- ★ Infrastructure as Code
- ★ Continuous Feedback

4. Traditional vs DevOps

Traditional Approach	Aspect	DevOps Approach
Siloed teams (Dev & Ops separate)	Team Structure	Cross-functional team (Dev + Ops together)
Manual processes	Process	Automated processes
Long release cycle (months)	Release Frequency	Short release cycle (days/hours)
Focus on individual goals	Focus	Focus on customer value
More errors and failures	Quality	Less errors, high quality
Reactive (fix after failure)	Monitoring	Proactive (monitor and improve)

5. Key Practices in DevOps

- **Continuous Integration (CI)**: Developers integrate code frequently into a shared repository.
- **Continuous Delivery (CD)**: Code is always in a deployable state and can be released anytime.
- **Infrastructure as Code (IaC)**: Manage infrastructure using code (e.g., Terraform, Ansible).
- **Automation**: Automate repetitive tasks like build, test, deploy, monitoring.
- **Monitoring & Logging**: Monitor application and infrastructure and collect logs for analysis.

6. Benefits of DevOps

- Faster time to market
- Improved collaboration
- Higher productivity
- Better quality of software
- Scalability and stability
- Reduced costs

DevOps is a Journey,
not a Destination. !!

7. DevOps Tools (Examples)

Version Control	CI/CD Tools	Build Tools	Containerization	Orchestration	IaC Tools	Monitoring Tools
Git, GitHub, GitLab, Bitbucket	Jenkins, GitLab CI, GitHub Actions, CircleCI	Maven, Gradle, Ant	Docker, Podman	Kubernetes, Docker Swarm	Terraform, Ansible, Puppet, Chef	Prometheus, Grafana, ELK Stack



DevOps - Linux Basics

Linux is the backbone of DevOps



What is Linux?

Linux is an open source, Unix-like operating system. It is widely used in servers, cloud, containers and DevOps environments because it is secure, stable and flexible. ★

1. Linux Distributions

Some popular Linux distributions used in DevOps:

- Ubuntu
- CentOS / AlmaLinux / Rocky Linux
- Debian
- Red Hat Enterprise Linux (RHEL)
- Amazon Linux

2. File System Structure

/ (root)	
/bin	= Essential command binaries
/sbin	= System binaries (admin cmds)
/etc	= Configuration files
/home	= User home directories
/var	= Variable files (logs, spool, cache)
/tmp	= Temporary files
/usr	= User programs and files
/opt	= Optional add-on applications
/boot	= Boot loader files
/dev	= Device files
/proc	= Process and kernel info (virtual)
/root	= Root user's home directory
/lib	= Essential shared libraries
/mnt	= Mount point for temporary mounts
/media	= Mount point for removable media
/srv	= Service data

3. Basic Commands

Command	Example	Description
pwd	pwd	Print working directory
ls	ls -l	List files and directories
cd	cd /home/user	Change directory
mkdir	mkdir dir1	Create a directory
rmdir	rmdir dir1	Remove empty directory
rm	rm file.txt	Remove a file
cp	cp src.txt dest.txt	Copy file
mv	mv old.txt new.txt	Move or rename file
cat	cat file.txt	Display file content
less	less file.txt	View file content page by page
head	head -n 10 file.txt	First 10 lines of file
tail	tail -n 10 file.txt	Last 10 lines of file
touch	touch file.txt	Create an empty file
which	which python3	Location of a command
man	man ls	Manual of a command
clear	clear	Clear terminal screen

8. User Management

• whoami	→ Current logged in user
• id	→ User and group id
• adduser username	→ Add a new user (Ubuntu/Debian)
• useradd username	→ Add a new user (RHEL/CentOS)
• passwd username	→ Set or change password
• usermod -aG group user	→ Add user to group
• userdel username	→ Delete a user

4. File Operations

Command	Description
ls -l	Long listing (with permissions, size, date)
ls -a	Show hidden files
ls -lh	Human readable sizes
cp -r dir1 dir2	Copy directory recursively
mv file1 file2	Rename file1 to file2
rm -r dir	Remove directory recursively
rm -f file	Force remove file
find /path -name "*.txt"	Find files by name
grep "text" file.txt	Search text in file
wc -l file.txt	Count lines in file

5. File Permissions

Permission Notation	Numeric Representation
- r w x r w x r w x	r w x = 4+2+1 = 7
user group others	r w - = 4+2 = 6
	r - x = 4+1 = 5
r = read (4)	r - - = 4
w = write (2)	- w x = 2+1 = 3
x = execute (1)	- - x = 1
	- - - = 0

chmod Examples

- chmod 755 file.sh → rwxr-xr-x
- chmod 644 file.txt → rw-r--r--
- chmod +x file.sh → add execute permission
- chmod -x file.sh → remove execute permission
- chown user:group file.txt → change owner and group

6. Process Management

- ps aux → Show all running processes
- top → Real-time process monitor
- htop → Interactive process viewer (if installed)
- kill <PID> → Kill process by PID
- kill -9 <PID> → Force kill process

7. System Info Commands

- uname -a → System and kernel info
- hostname → Show or set hostname
- df -h → Disk space usage
- du -sh /path → Size of directory
- free -h → Memory usage
- uptime → System uptime and load

9. Networking Commands

- ip a / ifconfig → Show IP address and interfaces
- ping google.com → Check connectivity
- netstat -tuln → Show open ports and listening services
- ss -tuln → Another way to show open ports
- curl google.com → Fetch response from URL
- wget https://site.com → Download file from URL

★ Tip: Practice daily in terminal. Linux is all about practice and exploration! 😊

Git - A Version Controlling Tool

1

1. Knowing about Version Control

- Version Control System (VCS) records changes to files over time so that you can recall specific versions later.
- It helps in collaboration, backup, tracking history, and reverting mistakes.
- Types: Centralized VCS (e.g., SVN)
Distributed VCS (e.g., Git)

Track
Changes
Collaborate
Better

Git is the most
popular Distributed
Version Control
System. ★

2. Git - A CLI

Git is a Command Line Interface (CLI) tool used to interact with Git repositories.

Why CLI?

- ✓ Fast and lightweight
- ✓ Powerful and flexible
- ✓ Works on all major OS
- ✓ The base for all Git GUIs

Basic CLI Syntax

git <command> <options> <arguments>

Example:

```
git status
git add file.txt
```

3. Essentials of Git in Industry

- ★ Collaboration in teams
- ★ Code backup and recovery
- ★ Track changes and history
- ★ Branching for features & releases
- ★ Integration with CI/CD pipelines
- ★ Supports DevOps & Agile workflows.

4. How to Setup Git

4.1 Installing Git

- Download from <https://git-scm.com/downloads>
- Install for your OS (Windows/macOS/Linux)
- Verify installation:

```
git --version
```

4.3 Getting a Git Repository

Create a New Repository

```
mkdir my-project
cd my-project
git init
```

(Creates a .git folder)

Clone an Existing Repository

```
git clone <repo-url>
cd <repo-name>
```

A repository (repo) is a project folder tracked by Git. ☺

5. Working with various commands in Git

2

git init	→ Initialize a new repository
git clone <url>	→ Clone an existing repository
git status	→ Check status of files
git add <file>	→ Add file to staging area
git commit -m "msg"	→ Commit changes with message
git log	→ Show commit history
git branch	→ List branches
git checkout <branch>	→ Switch branches
git merge <branch>	→ Merge branch into current
git remote -v	→ List remotes
git push	→ Push changes to remote
git pull	→ Pull changes from remote

6. Recording Changes to the Repository

- How to check the Status of Your Files
- How to track New Files
- Staging our modified files
- Ignoring Files from GIT

```
git status
git add <file>
git add .
```

Create a .gitignore file
Add patterns (e.g., *.log, node_modules/)
Git will ignore those files.

- Viewing Your Unstaged and Staged Changes
- How to commit Your Changes
- Skipping the Staging Area and commit
- Removing Files from GIT

```
git diff (unstaged)
git diff --staged (staged)
git commit -m "message"
git commit -am "message"
git rm <file>
```

7. Viewing the Commit History

git log → Full history

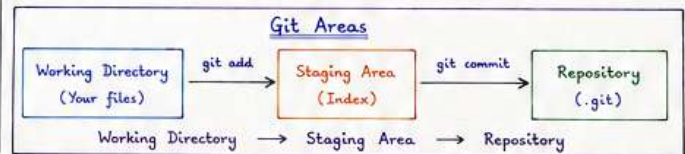
7.1 Limiting Log Output

```
git log --oneline → One line per commit
git log -n 5 → Last 5 commits
git log --since="7 days ago" → Commits in last 7 days
git log --author="name" → Commits by an author
```

7.2 Using a GUI to Visualize History

Tools: Git GUI, SourceTree, GitKraken, VS Code

These show branches, commits, diffs in a graphical way.



8. Undoing Things

8.1 Changing Your Last Commit

- If not pushed: `git commit --amend`
- Change message or add files to last commit.

8.2 Unstaging a Staged File

```
git restore --staged <file> (or) git reset HEAD <file>
```

8.3 Unmodifying a Modified File

```
git restore <file>
(Discards changes in working directory)
```

9. Working with Remotes

9.1 Showing Your Remotes

```
git remote -v
```

9.2 Adding Remote Repositories

```
git remote add origin <url>
```

9.3 Fetching and Pulling from Your Remotes

- `git fetch origin` → Download changes
- `git pull origin <branch>` → Fetch + Merge

9.4 Pushing to Your Remotes

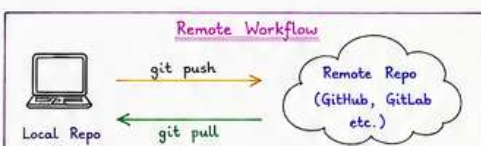
```
git push -u origin <branch>
```

9.5 Inspecting a Remote

```
git remote show origin
```

9.6 Removing and Renaming Remotes

- Remove: `git remote remove origin`
- Rename: `git remote rename origin upstream`



10. Branching and Merging in Git

3

10.1 What a Branch Is

A branch is a lightweight movable pointer to a commit. It allows you to work on different features independently.

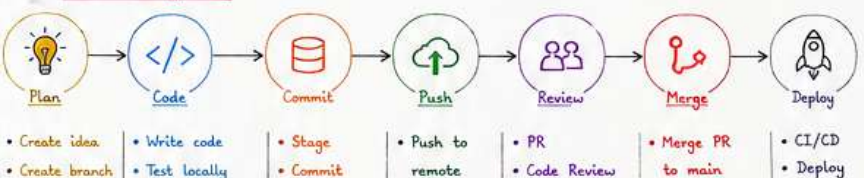
10.2 Basic in Branching and Merging

- `git branch <name>` → Create branch
- `git checkout <name>` → Switch branch
- `git checkout -b <name>` → Create & switch
- `git merge <branch>` → Merge into current branch

10.3 Branch Management in Git

```
git branch → List branches
git branch -d <name> → Delete branch (safe)
git branch -D <name> → Force delete branch
```

11. Git Workflows



Good Git practices lead to better collaboration, clean history and successful DevOps! ☺

★ Practice daily, explore more and make Git your super power! ☺



Automate
Build, Test
and Deploy
Repeatably! 😊

JENKINS

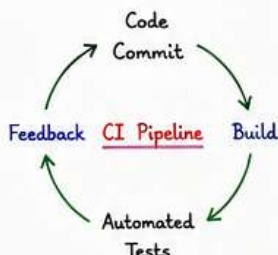
Jenkins is an open source automation server used to build, test and deploy applications continuously in a DevOps environment. ★

1. Essentials of Continuous Integration

Continuous Integration (CI) is a DevOps practice where developers integrate code into a shared repository frequently. Every integration triggers an automated build and test to detect issues early.

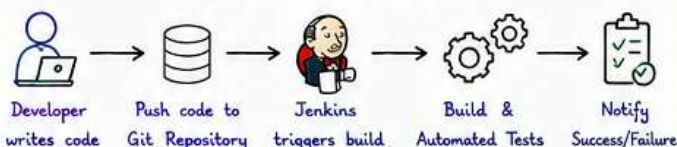
Key Benefits:

- Detects bugs early
- Improves code quality
- Reduces integration time and effort
- Increases developer productivity
- Provides faster feedback
- Enables frequent and reliable releases



2. An example scenario where CI is used

Example: E-commerce Application



Flow:

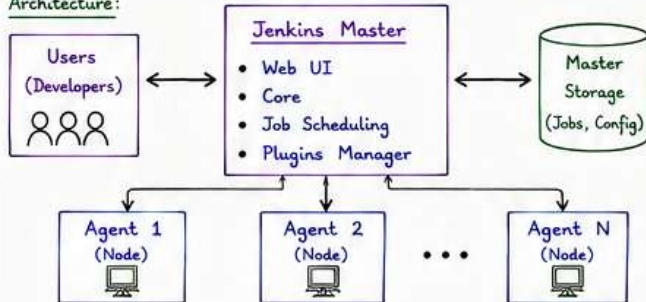
Code Commit → Jenkins Build → Automated Tests → Feedback

Benefits: Bugs found early, builds are reliable, team productivity increases & happy customers! 😊

3. Know about Jenkins and its Architecture in detail

Jenkins is a self-contained automation server written in Java. It works on a master-agent architecture.

Architecture:



- **Master:** Controls the overall operations, scheduling, configuration.
- **Agents (Nodes):** Execute the jobs/builds assigned by the master.
- **Plugins:** Extend functionality (SCM, Build tools, Notifications etc.)
- **Jobs:** Set of tasks defined to achieve a particular goal.

Jenkins stores all configuration, job data, logs etc. in \$JENKINS_HOME\$ directory.

6. Authentication

Authentication decides how users prove their identity. Jenkins supports multiple authentication methods.

Methods:

- Jenkins' own user database
- LDAP
- Active Directory
- OAuth / Google Login / GitHub Login
- Other OAuth providers

6.1 Jenkins own database user creation

Steps:

1. Manage Jenkins → Manage Users
2. Click on Create User
3. Enter Username, Password, Full Name, Email
4. Click Create User
5. User is stored in Jenkins' internal database



Choose authentication method as per organization policy and security requirements. 😊

4. Jenkins Tool Management in detail

Jenkins can automatically install and manage tools required for builds.

Managed Tools:

- JDK (Java Development Kit)
- Maven
- Gradle
- Git
- Ant
- Docker
- NodeJS
- And many more...

Configuration Steps

1. Manage Jenkins
2. Global Tool Configuration
3. Select tool type
4. Add installation path or Let Jenkins auto-install (if supported)
5. Save



5. Know about User management in Jenkins

Jenkins allows us to manage different users and provide them appropriate access.

Types of Users:

- Admin Users : Manage Jenkins system.
- Regular Users : Create and run jobs.
- Read-only Users : View jobs and builds.

User Management Steps:

1. Manage Jenkins → Manage Users
2. Create User / Configure User
3. Set password and other details
4. Assign roles/permissions
5. Save



7. Authorization

Authorization defines what actions a user is allowed to perform in Jenkins. Jenkins supports different authorization strategies.

7.1 Matrix based authorization

- Permissions are assigned globally for the whole instance.
- Fine-grained control at global level.

Steps:

1. Manage Jenkins → Configure Global Security
2. Select "Matrix-based security"
3. Define Global Permissions (Overall/Administer etc.)
4. Save

Example: Global Permissions

User	Overall	Read	Run Scripts	Configure	Administer
admin	✓	✓	✓	✓	✓
dev1	✓	✓	✓	✗	✗
tester	✓	✓	✓	✗	✗

7.2 Project based authorization

- Permissions are defined at Job/Folder/Item level.
- Different permissions for different projects.

Steps:

1. Create or open a Job
2. Click "Configure" → "Security"
3. Select "Project-based Matrix Authorization Strategy"
4. Add permissions for users/groups
5. Save

Example: Job Level Permissions

Job: Build-App					
User	Build	Configure	Cancel	Read	
dev1	✓	✓	✓	✓	
tester	✓	✓	✗	✓	
intern	✓	✓	✗	✗	

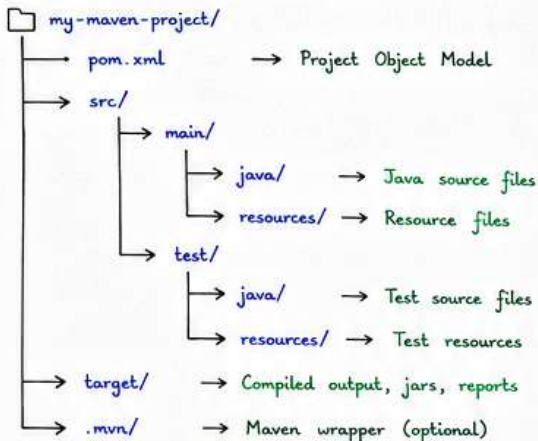
8. Summary

- ★ CI helps integrate code frequently and catch issues early.
- ★ Jenkins automates build, test and deployment using master-agent architecture.
- ★ Tools can be managed automatically to simplify builds.
- ★ Users are managed with proper roles and permissions.
- ★ Authentication verifies WHO the user is.
- ★ Authorization controls WHAT the user can do.

Jenkins
makes DevOps
possible with
Automation! 😊

1. Maven Project Structure

Maven follows a standard directory layout which helps in maintaining consistency across projects.



Conventions → Configuration
(Write less XML, do more coding!)

2. Maven Plugins

Plugins are the heart of Maven. Each plugin performs a well-defined task (compile, test, package, deploy, etc.).

Some Common Plugins:

Plugin	Purpose
maven-compile-plugin	Compiles the source code
maven-surefire-plugin	Runs unit tests
maven-jar-plugin	Creates JAR file
maven-war-plugin	Creates WAR file
maven-install-plugin	Installs artifact to local repo
maven-deploy-plugin	Deploys artifact to remote repo

Plugins are bound to lifecycle phases to achieve goals.

3. Project Object Model (POM) – Fundamental Unit of Work in Maven Project

POM (pom.xml) is the core of Maven project. It contains project information, build configuration, dependencies, plugins, etc.

Basic Structure of pom.xml:

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>      → Group / Organization
  <artifactId>my-maven-project</artifactId> → Project Name
  <version>1.0.0</version>             → Project Version
  <packaging>jar</packaging>           → Packaging Type (jar/war)
  <name>My Maven Project</name>        → Project Name
  ... (dependencies, plugins, properties, etc.)
</project>
  
```

★ Key Points:

- POM is the single source of truth for the project.
- Maven reads POM to build the project and manage dependencies.

4. Maven Build Lifecycle

A lifecycle is a sequence of phases. Each phase has a specific goal. Maven has 3 built-in lifecycles.

Lifecycle	Purpose	Phases (in order)
clean	Cleans the project	pre-clean, clean, post-clean
default (build)	Builds and deploys the project	validate, initialize, generate-sources, process-sources, generate-resources, process-resources, compile, process-classes, generate-test-sources, process-test-sources, generate-test-resources, process-test-resources, test-compile, process-test-classes, test, prepare-package, package, verify, install, deploy
site	Generates project documentation	pre-site, site, post-site, site-deploy

Default Lifecycle Flow (Simplified)



Key Phases:

- validate - Validates the project is correct and all necessary information is available.
- compile - Compiles the source code.
- test - Runs unit tests using a suitable framework.
- package - Packages the compiled code in distributable format (jar/war).
- install - Installs the package into local repository.
- deploy - Copies the final package to remote repository.

5. Adding External Dependencies to maven pom.xml

Dependencies are libraries required for the project.

Add inside <dependencies> section:

```

<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>    <!-- optional -->
  </dependency>
</dependencies>
  
```

Scopes:

- compile (default) - Available at compile and runtime
- provided - Provided by JDK / container at runtime
- runtime - Needed only at runtime
- test - Only for test classes
- system - Uses a system provided jar

Notes:

- Transitive dependencies are handled automatically by Maven.
- Use "mvn dependency:tree" to view dependency tree.

6. Maven Build and Test Project

Common Maven Commands (using CLI):

Command	Purpose
mvn clean	Cleans the target directory
mvn compile	Compiles the source code
mvn test	Runs unit tests
mvn package	Packages the project (jar/war)
mvn install	Installs the package to local repository
mvn deploy	Deploys the package to remote repository
mvn clean install	Clean, build, test and install
mvn clean deploy	Clean, build, test and deploy

Example Build Flow:

```

mvn clean install → Code Compile → Tests Run → Package Created
                  → Installed in Local Repository
  
```




JENKINS

Creating Jobs and Automatic Build Settings

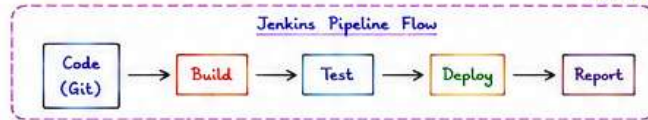
Jenkins is an open source automation server used to automate the build, test and deployment of applications. !!

1. What is Jenkins Pipeline?

A Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins.

Pipeline as Code : Define your delivery pipeline in a "Jenkinsfile" and keep it in source control.

Jenkinsfile



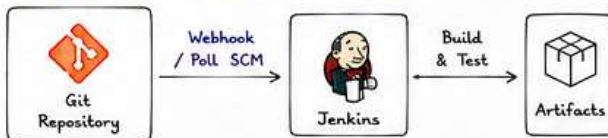
2. Why Pipeline?

- Automates the entire delivery process.
- Consistent and repeatable builds.
- Easy integration with SCM & other tools.
- Supports complex workflows.
- Pipeline as Code - versioned and reviewable.

Write once
Run anywhere! !!

3. Integration with GIT

- Pull source code from Git repository.
- Detect changes and trigger builds automatically.
- Supports Webhooks for instant build triggers.



4. How to enable project based authorization for a job

Steps:

- Go to Manage Jenkins → Configure Global Security
- Check "Enable project based matrix authorization"
- Save
- Go to the specific job → Configure → Permissions
- Set permissions for users/roles for this job only.

Permissions Example					
User/Role	Build	Configure	Read	Delete	Workspace
dev	✓	✓	✓	✗	✗
tester	✓	✗	✓	✗	✗
deployer	✓	✓	✓	✓	✓

5. Source Code Management while creating jobs

Steps:

- New Item → Enter job name → Freestyle / Pipeline job
- Scroll to "Source Code Management"
- Select "Git"
- Enter Repository URL (e.g., <https://github.com/user/repo.git>)
- Credentials (if private repo)
- Branch Specifier (e.g., */main)
- Save

SCM Options

- Git
- GitHub
- Bitbucket
- Subversion
- CVS

6. Triggering Automated Build

Trigger options:

- Build periodically (cron expression)
- Poll SCM
- GitHub Hook Trigger for GITscm polling
- Build after other projects are built
- Build when a change is pushed to Git (Using Webhook)

Example (Cron)

H/15 * * * *
→ Build every 15 min

Maven Goal Examples

- clean
- compile
- test
- package
- install

7. Maven Job Setup

Steps:

- Create a Freestyle project
- SCM → Git (add repo details)
- Build Triggers → Choose required trigger
- Build → Add build step → Invoke top-level Maven targets
 - Goals : clean install (or as required)
- Save

Jenkins automates your build, test and deployment process and makes DevOps implementation easy and reliable! !!

8. Post Build Options

Post Build actions are executed after the build is complete.

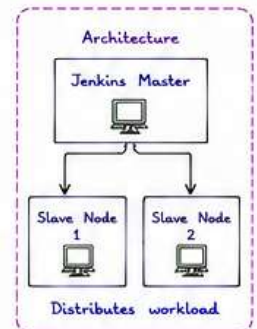
Common post build options:

Option	Description
Email Notification	Send email on build success/failure.
Trigger another build	Trigger downstream job after completion.
Publish JUnit test result	Publish test reports (JUnit XML).
Publish HTML reports	Publish HTML reports.
Archive the artifacts	Save artifacts generated by the build.
Slack / MS Teams Notification	Send build status to Slack / Teams.
Cobertura / JaCoCo Report	Publish code coverage reports.
Set build status	Update build status (Stable/Unstable).

9. Adding a Slave Node to Jenkins

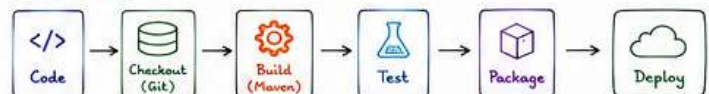
Steps:

- Manage Jenkins → Manage Nodes and Clouds
- Click "New Node"
- Enter Node Name
- Select "Permanent Agent"
- Set Number of Executors
- Set Remote Root Directory
- Launch method (Launch agent via JNLP / SSH)
- Save
- Start the Node



10. Building Delivery Pipeline

Example Flow:



Sample Jenkins Pipeline (Jenkinsfile)

```

pipeline {
    agent any
    stages {
        stage('Checkout') { steps { git 'https://github.com/user/repo.git' } }
        stage('Build') { steps { sh 'mvn clean install' } }
        stage('Test') { steps { sh 'mvn test' } }
        stage('Package') { steps { sh 'mvn package' } }
        stage('Deploy') { steps { sh './deploy.sh' } }
    }
    post {
        always { mail to: 'devteam@company.com', subject: 'Build Status',
            body: 'Pipeline completed' }
    }
}
  
```

11. Notification Settings in Jenkins

Methods:

- Email Notification
 - Manage Jenkins → Configure System → Extended E-mail Notification
 - Add SMTP server details, test configuration, save
 - Use in job Post-build actions
- Slack / MS Teams Notification (Using Plugins)
- Build status on dashboard

Email Triggers

- Always
- Failure
- Unstable
- Success
- Not built

12. Plugin Management in Jenkins

Managing Plugins:

- Manage Jenkins → Manage Plugins
- Available : List of plugins to install
- Installed : List of installed plugins
- Updates : Available plugin updates
- Advanced : Upload .hpi file, Update Site

Useful Plugins

- Pipeline
- Git
- Maven Integration
- Publish Over SSH
- Email Extension
- Slack Notification
- JUnit
- SonarQube Scanner



Happy Automating!



DOCKER

Docker is a platform that uses containerization technology to build, ship and run applications consistently in any environment. 😊

1. INTRODUCTION

1.1 Real-world Shipping Transportation Challenges

Imagine a shipping company that delivers goods from one place to another.

- Different sizes of trucks, ships, and planes.
- Goods are packed manually → More time & labor.
- Risk of damage or loss during transport.
- Different rules at each port → Delays.
- Hard to track and manage all shipments.
- High cost and inefficient process.



1.2 Introducing Docker and its Technology

- Docker solves these challenges using containerization.
- It packages an application and all its dependencies into a single unit called a container.
- The same container can run anywhere - on a laptop, server, or cloud.



Benefits:

- ★ Lightweight and fast
- ★ Consistent across environments
- ★ Easy to deploy and scale
- ★ Better resource utilization

1.3 Understanding Docker Images and Containers

Docker Image: A read-only template that contains the application code, runtime, libraries, environment variables and configuration files.

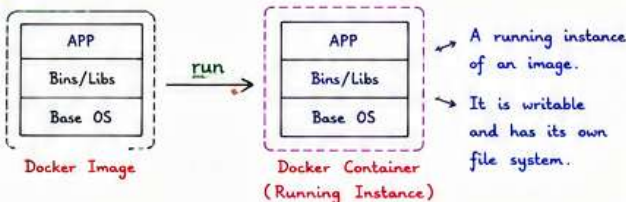


Image	Container
Template (Read-only)	Running Instance (Read-Write)
Created using Dockerfile	Created from Image
Can be shared	Ephemeral (Can be stopped/removed)
Stored in Docker Hub	Runs on Docker Engine

Key Takeaway:

Image is like a blueprint of a house.
Container is the actual house built from that blueprint.



Summary:

- ★ Docker solves real-world shipping challenges by containerizing applications.
- ★ Images are templates, Containers are running instances.
- ★ Containers go through a life cycle: Create → Start → Running → Stop → Remove.
- ★ We can customize base images, create Dockerfiles and publish images to Docker Hub for sharing and reuse.

2. WORKING WITH CONTAINER

2.1 How to Share and Copy a Container

- You can create a new image from an existing container and share it with others.

Steps:



Commit a container to an image

```
docker commit <container_id> <image_name>:<tag>
```

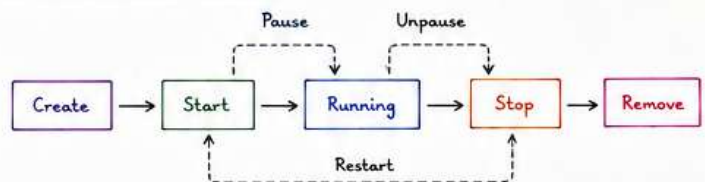
Tag the image (optional)

```
docker tag <image_name>:<tag> <dockerhub_username>/<image_name>:<tag>
```

Push to Docker Hub

```
docker push <dockerhub_username>/<image_name>:<tag>
```

2.2 Container Life Cycle



Common Commands:

docker create <image>	→ Create a container
docker start <container>	→ Start a container
docker pause <container>	→ Pause a running container
docker unpause <container>	→ Unpause a container
docker stop <container>	→ Stop a container
docker restart <container>	→ Restart a container
docker rm <container>	→ Remove a stopped container

2.3 How to use Base Image and Customize

- A base image is a starting point. We customize it by adding our application, libraries, configs, etc.

Example: Using ubuntu as base image and installing Nginx

```
FROM ubuntu:22.04 # Base Image
RUN apt-get update && apt-get install -y nginx # Customize
COPY index.html /var/www/html/index.html # Copy file
EXPOSE 80 # Expose port
CMD ["nginx", "-g", "daemon off;"] # Run command
```

2.4 Creation of Docker File

- A Dockerfile contains instructions to build a Docker image.

Basic Dockerfile Example:

```
FROM python:3.11-slim # Base Image
WORKDIR /app # Set working directory
COPY requirements.txt # Copy dependency file
RUN pip install -r requirements.txt # Install dependencies
COPY . # Copy source code
EXPOSE 5000 # Expose port
CMD ["python", "app.py"] # Run application
```

2.5 How to Publish Image on Docker Hub

Steps:

1. Create account on hub.docker.com
2. Login using CLI → `docker login`
3. Tag your image → `docker tag <image> <username>/<image>:<tag>`
4. Push image → `docker push <username>/<image>:<tag>`

Local Image



Push



Key Benefits of Docker

- ✓ Consistency
- ✓ Scalability
- ✓ Portability
- ✓ Version Control
- ✓ Efficiency
- ✓ Easy Collaboration





INTRODUCTION TO DOCKER NETWORKING

Docker uses containerization technology to build, ship and run applications consistently in any environment. ☺

1. Network Types in Docker Technology

Docker provides several network drivers. Here are the default network types:

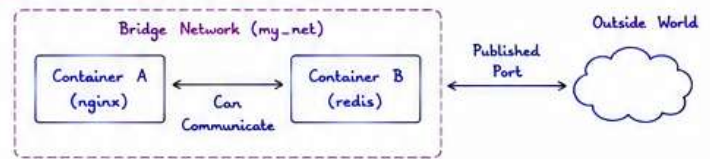
Network Type	Driver	Description	Use Case
<u>bridge</u>	bridge	Default network. Containers in the same bridge network can communicate.	Single-host communication
<u>host</u>	host	Container shares the host's network namespace.	High performance applications
<u>none</u>	null	No networking for the container.	Isolated containers
<u>overlay</u>	overlay	Connects multiple Docker daemons together.	Multi-host communication (Swarm)
<u>macvlan</u>	macvlan	Assigns a MAC address to a container, making it appear as a physical device on the network.	Legacy applications that expect physical network
<u>ipvlan</u>	ipvlan	Similar to macvlan but more efficient.	Large scale deployments

Use: Create isolated or shared networks between containers or between containers and outside world.

2. Docker Container Networking

- By default, every container is connected to the bridge network.
- Containers on the same network can communicate using container name.
- Containers on different networks cannot talk to each other unless explicitly allowed.
- You can create custom networks to isolate and connect containers.

Example: Communication in Bridge Network



Useful Commands:

- `docker network ls` → List all networks
- `docker network create my_net` → Create a custom network
- `docker network inspect my_net` → Inspect network details
- `docker run --network my_net nginx` → Run container in a network
- `docker network connect my_net container` → Connect running container
- `docker network disconnect my_net container` → Disconnect container

3. Docker Compose - An Introduction

- Docker Compose is a tool for defining and running multi-container Docker applications.
- It uses a YAML file (`docker-compose.yml`) to configure application services.
- With a single command, you can create and start all services.

Sample docker-compose.yml (for php, wordpress, mysql)

```

version: '3.8'
services:
  db:
    image: mysql:5.7           # MySQL Service
    restart: always           # Auto restart
    environment:               # Environment variables
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
    volumes:                   # Persist data
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest    # WordPress Service
    restart: always
    depends_on:                 # Depends on db
      - db
    ports:                       # Port mapping
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wordpress
      # Connect to MySQL

  phpmyadmin:
    image: phpmyadmin/phpmyadmin # PHPMysql Service
    restart: always
    ports:                         # Port mapping
      - "8081:80"
    environment:
      PMA_HOST: db
      MYSQL_ROOT_PASSWORD: rootpass

volumes:
  db_data:                       # Named volume
  
```

Useful Commands:

- `docker-compose up -d` → Start all services in background
- `docker-compose down` → Stop and remove containers, networks, volumes
- `docker-compose ps` → List running services
- `docker-compose logs -f` → View logs
- `docker-compose build` → Rebuild services

Benefits of Compose:

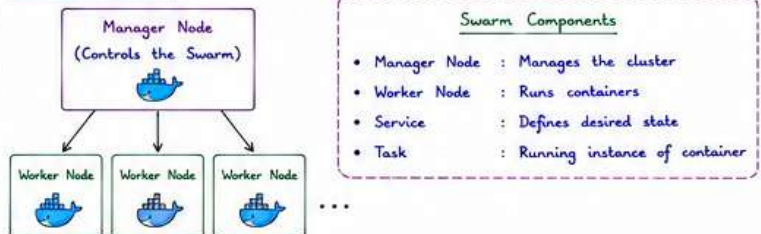
- ✓ Easy configuration
- ✓ Reproducible environments
- ✓ Manage multi-container applications
- ✓ Works in development, test & production



4. Docker Swarm - An Introduction

- Docker Swarm is Docker's native clustering and orchestration tool.
- It turns a pool of Docker hosts into a single virtual Docker engine.
- It provides: High availability, Scaling, Load balancing, Self-healing.

Swarm Architecture



5. Use Docker Compose to create php, wordpress, mysql

We can use the `docker-compose.yml` (shown in section 3) file.

Commands:

- `docker-compose up -d` → Creates and starts php, wordpress, mysql
- Access WordPress: `http://<host-ip>:8080`
- Access phpMyAdmin: `http://<host-ip>:8081`

6. How to Start Containers on a Cluster with Docker Swarm

Steps:

- Initialize Swarm on Manager Node
`docker swarm init`
- Join Worker Nodes to the Swarm
`docker swarm join --token <token> <manager-ip>:2377`
- Verify Nodes
`docker node ls`
- Deploy a Service (from Compose file)
`docker stack deploy -c docker-compose.yml myapp`

Check Swarm Status

```

docker info
docker node ls
docker service ls
  
```

7. Creating and Scaling an Application in Docker Swarm

Create Application (Stack)

`docker stack deploy -c docker-compose.yml myapp`

This creates all services defined in compose file as swarm services.

List Services ☺

```

docker service ls
docker service ps
<service_name>
  
```

Scaling Services

`docker service scale myapp_wordpress=3`



Other Useful Commands

- `docker service scale myapp_wordpress=5`
- `docker service update --image wordpress:latest myapp_wordpress`
- `docker stack rm myapp`

Summary: Docker Networking allows containers to communicate securely. Docker Compose helps run multi-container apps easily. Docker Swarm helps deploy, scale and manage apps in cluster environments.

Happy Learning! ☺



KUBERNETES

Automating containerized applications at scale

Kubernetes (K8s) is an open-source container orchestration platform that automates deployment, scaling, and management of containerized applications. ☺

1. Introduction to Kubernetes

Kubernetes (K8s) is an open-source system for automating deployment, scaling, and management of containerized applications.

Key Features :

- Automated deployment and rollbacks
- Horizontal scaling and self-healing
- Service discovery and load balancing
- Efficient resource utilization
- Declarative configuration (desired state)
- Portability across environments (cloud/on-prem)

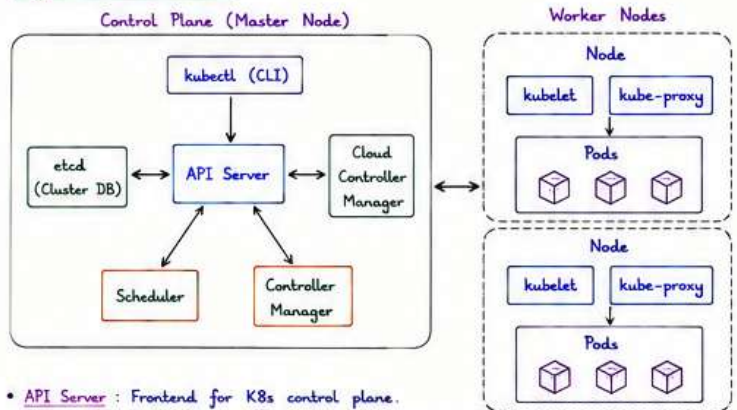
K8s vs Docker

Docker → Container Runtime
(creates and runs containers)

Kubernetes → Container Orchestrator
(manages containers at scale)

2. Kubernetes Cluster Architecture – An overview

High Level Architecture :



- **API Server** : Frontend for K8s control plane.
- **etcd** : Consistent and highly-available key-value store.
- **Scheduler** : Schedules pods to worker nodes.
- **Controller Manager** : Runs controller processes (ReplicaSet, Node, Endpoints etc.)
- **kubelet** : Ensures containers are running in a pod.
- **kube-proxy** : Network proxy, maintains network rules on nodes.

3. Understanding concepts of Pods, ReplicaSets, Deployments and Namespaces

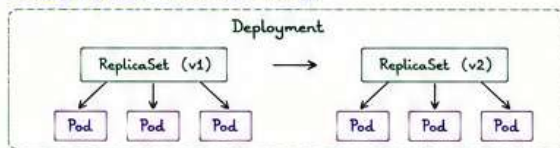
➤ **Pod** : Smallest deployable unit in K8s. A pod can have one or more containers that share the same network and storage.



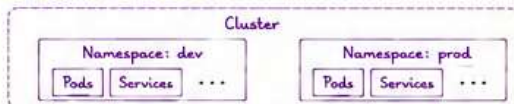
➤ **ReplicaSet** : Ensures that a specified number of pod replicas are running at any time.



➤ **Deployment** : Manages ReplicaSets and provides declarative updates (rolling update, rollback, scaling).



➤ **Namespace** : Provides a way to divide cluster resources between multiple users or teams.



5. Persistent Volumes and Persistent Volume Claims – an overview

Kubernetes provides persistent storage using PV and PVC.

➤ **Persistent Volume (PV)** : A piece of storage in the cluster provisioned by admin or dynamically.

PV
(NFS / EBS / GCE PD / Local / etc.)

➤ **Persistent Volume Claim (PVC)** : A request for storage by a user.

PVC
(Request for storage)

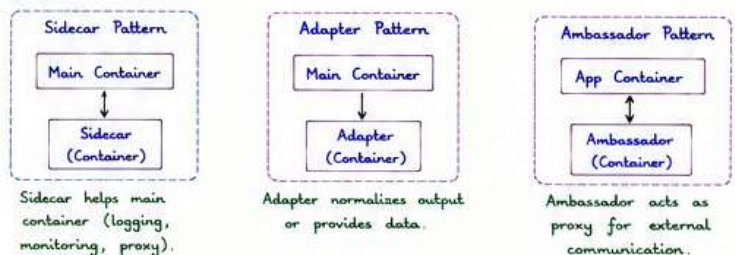


PVC acts as a request and binds to PV. Pod uses the claimed storage for data persistence even after restart.

6. Design of Pods

- Pods should be designed to run one main process per container (best practice).
- Use multi-container pods only when containers are tightly coupled.

Patterns :



7. Understanding labels, selectors, jobs, and schedulers

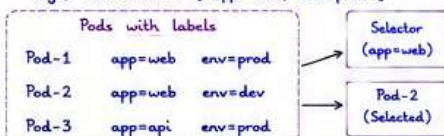
➤ **Labels and Selectors**

Labels : Key-Value pairs attached to objects.

e.g., app=web, env=prod, tier=frontend

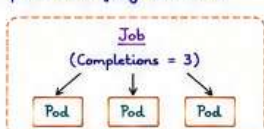
Selectors : Used to filter resources based on labels.

e.g., matchLabels: { app: web, env: prod }



➤ **Jobs**

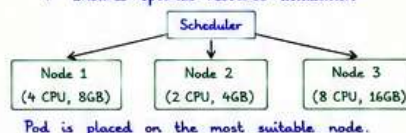
Jobs create one or more pods and ensure that a specified number of pods successfully terminate.



➤ **Scheduler**

Scheduler assigns pods to nodes based on resources, constraints and policies.

- ✓ Considers CPU, Memory, GPU, Disk
- ✓ Node affinity / anti-affinity rules
- ✓ Taints and tolerations
- ✓ Ensures optimal resource utilization



Key Takeaways

- ★ Kubernetes automates container operations.
- ★ Declarative and desired state based.
- ★ Scalable, Portable and Resilient.
- ★ Rich ecosystem and Large community support.



ANSIBLE

ANSIBLE

Automation Made Simple! ☺

Ansible is an open-source automation tool used for configuration management, application deployment, task automation and orchestration. ☺

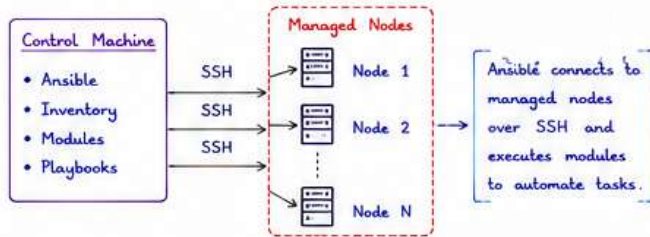
1. Introducing Ansible - A Configuration Management Tool

Ansible is agentless, easy to learn and uses simple YAML syntax for automation.

◎ Basics / What Will Be Installed

- Ansible (on control machine)
- Python (required on both control and managed nodes)
- SSH (for communication)
- No agent installation on managed nodes.

◎ Understanding Ansible Architecture



◎ Control Machine Requirements

- OS : Linux / macOS / Windows (WSL recommended)
- Python : 3.8 or higher
- Ansible : Latest stable version
- SSH access to managed nodes
- Sufficient disk space and memory

◎ Managed Node Requirements

- OS : Any (Linux, Windows)
- Python : 2.7+ or 3.5+ (pre-installed)
- SSH server must be running (Linux)
- WinRM enabled (Windows)
- User with sudo/Administrator privileges

2. Inventory

Inventory defines the list of hosts and groups that Ansible manages.

Sample Inventory File (hosts.ini)

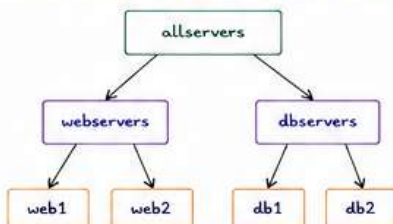
```
[webservers]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11

[dbservers]
db1 ansible_host=192.168.1.20
db2 ansible_host=192.168.1.21

[allservers:children]
webservers
dbservers
```

Hosts and Groups

- Hosts are individual machines.
- Groups are a collection of hosts.
- Children groups allow nesting of groups.



Notes:

- Ansible is agentless and uses SSH/WinRM for communication.
- Inventory is the backbone of Ansible automation.
- Modules perform tasks; Playbooks combine tasks and modules in YAML.

Host Variables

- Variables specific to a particular host.

```
# host-vars/web1.yml
ansible_user: ubuntu
ansible_port: 22
app_env: production
```

→ This file contains variables only for web1 host.

Group Variables

- Variables that apply to all hosts in a group.

```
# group-vars/webservers.yml
http_port: 80
max_clients: 200
app_env: staging
```

→ These variables will be available to all hosts in webservers group.

3. Learn Various Ansible Modules

Modules are the building blocks of Ansible. They are used to perform specific tasks on managed nodes.

Category	Module	Purpose	Example Usage
System Modules	ping	Check connectivity	ansible all -m ping
	command	Run commands on remote nodes.	ansible all -m command -a "uptime"
	shell	Run shell commands	ansible all -m shell -a "df -h"
	raw	Run commands without module subsystem	ansible all -m raw -a "whoami"
	setup	Gather facts about hosts	ansible all -m setup
File Modules	copy	Copy files to remote nodes	ansible web -m copy -a "src=index.html dest=/var/www/html/"
	file	Manage files and directories	ansible all -m file -a "path=/tmp/test state=absent"
	template	Deploy templates (Jinja2)	ansible all -m template -a "src=app.j2 dest=/opt/app.conf"
Package Modules	apt	Manage apt packages (Debian/Ubuntu)	ansible all -m apt -a "name=nginx state=present"
	yum	Manage yum packages (RHEL/CentOS)	ansible all -m yum -a "name=httpd state=latest"
Service Modules	service	Manage services	ansible all -m service -a "name=nginx state=started"
	systemd	Manage systemd services	ansible all -m systemd -a "name=nginx enabled=yes"
User Modules	user	Manage user accounts	ansible all -m user -a "name=devops state=present"
	group	Manage groups	ansible all -m group -a "name=devops state=present"
Other Useful Modules	git	Clone/manage git repositories	ansible all -m git -a "repo=https://github.com/..."
	cron	Manage cron jobs	ansible all -m cron -a "name='Backup' minute=0 hour=2 job='/script.sh'"
	firewalld	Manage firewall rules	ansible all -m firewalld -a "service=http state=enabled"
	script	Run local scripts on remote nodes	ansible all -m script -a "backup.sh"
	lineinfile	Ensure line in file is present	ansible all -m lineinfile -a "path=/etc/hosts line='127.0.0.1 local'"

Sample Ad-hoc Commands

- ansible all -m ping
- ansible webservers -m service -a "name=nginx state=restarted"
- ansible dbservers -m yum -a "name=mysql state=present"

Automate Everything, Repeat Nothing! ☺



ANSIBLE

ANSIBLE - How to use Ad-hoc Commands

Ad-hoc commands are one-line commands used to perform tasks without writing playbooks. 😊

1. How to use Ad-hoc Commands

- Ad-hoc commands are executed using the `ansible` command.
- Syntax : `ansible <pattern> -m <module> -a "<arguments>" [options]`
- pattern can be hostname, group name, all or IP.
- Common options : `-i` (inventory), `-u` (user), `-k` (ask password), `-b` (become sudo)

Examples :

Command	Description
<code>ansible all -m ping</code>	Check connectivity to all hosts
<code>ansible webserver -m command -a "uptime"</code>	Run uptime command on webserver group
<code>ansible db1 -m shell -a "df -h"</code>	Run shell command on host db1
<code>ansible all -m setup</code>	Gather facts about all hosts
<code>ansible all -m ping -i inventory.ini -u ansibleuser -k</code>	Ping all hosts using specific inventory, user and ask password

2. Parallelism

- Ansible executes tasks in parallel (default 5 forks).
- You can control the number of parallel connections using forks in `ansible.cfg` or `-f` option.

Ways to Control Parallelism :

- `ansible.cfg` (default)
`forks = 5`
- Command line
`ansible all -m ping -f 10 # Run with 10 parallel connections`

- Check current forks setting:
`ansible --version | grep forks`

3. Shell Commands

- Use `command` module for simple commands (no shell features).
- Use `shell` module when command needs shell features like pipes, redirection, variables, etc.

command Module (recommended)	shell Module
<code>ansible all -m command -a "ls -l /tmp"</code> - Faster and more secure - Does not invoke shell	<code>ansible all -m shell -a "cat /etc/passwd wc -l"</code> - Supports pipes, redirects, variables etc. - Slower and may have shell risks

4. File Transfer

- Use `copy` module to copy files to remote hosts.

Examples :

Command	Description
<code>ansible webserver -m copy -a "src=/etc/hosts dest=/tmp/hosts"</code>	Copy file to remote host
<code>ansible webserver -m copy -a "content='Hello Ansible' dest=/tmp/hello.txt"</code>	Create file with given content
<code>ansible webserver -m fetch -a "src=/var/log/syslog dest=./logs/flat=yes"</code>	Fetch file from remote to local machine

5. Managing Packages

- Use `package` module (auto detects package manager) or specific modules.

Examples :

Command	Description
<code>ansible all -m package -a "name=nginx state=present"</code>	Install nginx
<code>ansible all -m package -a "name=nginx state=absent"</code>	Remove nginx
<code>ansible all -m package -a "name=nginx state=latest"</code>	Upgrade nginx
<code>ansible all -m yum -a "name=httpd state=present"</code>	Install httpd using yum (RHEL)
<code>ansible all -m apt -a "name=git state=present update-cache=yes"</code>	Install git using apt (Ubuntu)

6. Users and Groups

- Use `user` and `group` modules to manage users and groups.

Examples :

Command	Description
<code>ansible all -m user -a "name=devops state=present"</code>	Create user devops
<code>ansible all -m user -a "name=devops state=absent remove=yes"</code>	Delete user devops and remove home dir
<code>ansible all -m group -a "name=devops state=present"</code>	Create group devops
<code>ansible all -m user -a "name=devops group=devops append=yes shell=/bin/bash"</code>	Add user devops to group devops

7. Deploying From Source Control (Git)

- Use `git` module to deploy or update application code from repositories.

Examples :

Command	Description
<code>ansible webserver -m git -a "repo=https://github.com/user/app.git dest=/var/www/app version=main"</code>	Clone repo to destination
<code>ansible webserver -m git -a "repo=https://github.com/user/app.git dest=/var/www/app update=yes force=yes"</code>	Pull latest changes
<code>ansible webserver -m git -a "repo=https://github.com/user/app.git dest=/var/www/app version=v1.0.0"</code>	Checkout specific tag/branch

8. Managing Services

- Use `service` (or `systemd`) module to manage services.

Examples :

Command	Description
<code>ansible all -m service -a "name=nginx state=started"</code>	Start nginx service
<code>ansible all -m service -a "name=nginx state=stopped"</code>	Stop nginx service
<code>ansible all -m service -a "name=nginx state=restarted"</code>	Restart nginx service
<code>ansible all -m service -a "name=nginx state=reloaded"</code>	Reload nginx service
<code>ansible all -m service -a "name=nginx enabled=yes"</code>	Enable nginx on boot
<code>ansible all -m service -a "name=nginx enabled=no"</code>	Disable nginx on boot

Useful Tips

- ★ Use `-b` to run with sudo privileges.
- ★ Use `-k` to prompt for password.
- ★ Use `--limit` to target specific hosts.

Example :

- `ansible webserver --limit web1 -m ping`
- `ansible all -b -m package -a "name=vim state=present"`



Key Takeaways

- ✓ Ad-hoc commands are quick and powerful.
- ✓ Combine modules with options for automation without playbooks.
- ✓ Use the right module for the task.
- ✓ Perfect for quick checks, system maintenance and automation tasks.





INTRODUCTION TO YAML SCRIPT

YAML is a human-readable data serialization format widely used for configuration files, data exchange and automation. 😊

YAML : YAML Ain't Markup Language 😊

1. What is YAML?

- YAML is a readable, writeable format for data.
- It is a superset of JSON and is designed to be easy for humans to read and write.
- It is commonly used in DevOps tools like Ansible, Kubernetes, Docker Compose, CI/CD pipelines etc.
- File extension : .yaml or .yml

2. Key Features

- Human readable
- Easy structure using indentation
- Supports comments
- Language independent
- Widely used in configuration management

Example Use-cases

- ★ Ansible Playbooks
- ★ Kubernetes Manifests
- ★ Docker Compose files
- ★ CI/CD Pipeline configs

3. Basic YAML Syntax Rules

- YAML is indentation sensitive (spaces, not tabs).
- Use consistent indentation (2 spaces is recommended).
- Key : Value pairs are separated by a colon (:).
- Use - for lists (sequences).
- Comments start with #.

Example :

```
# This is a comment
name: Anjan Kumar
age: 25
city: Patna
skills:
  - DevOps
  - Python
  - Docker
is_student: false
marks:
  math: 90
  science: 85
```

- Comment
- Key : Value (String)
- Key : Value (Integer)
- List (Sequence)
- Boolean value
- Nested Mapping (Dictionary)

4. Data Types in YAML

Data Type	Example	Description
String	name: Anjan	Text value
Integer	age: 25	Whole number
Float	price: 99.99	Decimal number
Boolean	is_active: true	true / false
Null	value: null	Empty value
List	items: - apple - banana	Ordered collection
Dictionary	person: - name: Anjan - age: 25	Key : Value pairs

5. Comments in YAML

- Use # to add comments.
- Comments are ignored by YAML parser.

Example :

```
# User information
user:
  name: Anjan      # User name
  age: 25          # User age
  active: true     # Is user active?
```

6. Collections in YAML

1) List (Sequence)

- Defined using - before each item.

Example :

```
fruits:
  - apple
  - banana
  - orange
```

2) Dictionary (Mapping)

- Defined using key : value pairs.

Example :

```
person:
  name: Anjan
  age: 25
  city: Patna
```

3) Nested Structure

Example :

```
company:
  name: TCS
  location: India
  employees:
    - name: Anjan
      role: DevOps Engineer
    - name: Rohit
      role: Software Engineer
```

→ Dictionary (Mapping)

→ List (Sequence)

→ Dictionary inside List

7. Multi-line Strings

- Use | for literal block scalar (keeps new lines).
- Use > for folded block scalar (new lines replaced with spaces).

Using | (Literal)

```
message: |
  This is a multi-line
  string.
  It preserves
  all new lines.
```

Using > (Folded)

```
message: >
  This is a multi-line
  string.
  It folds new lines
  into spaces.
```

8. YAML in Ansible (Example Playbook)

```
---
- name: Install and start Nginx
  hosts: webservers
  become: yes
  tasks:
    - name: Install Nginx
      yum:
        name: nginx
        state: present
    - name: Start Nginx Service
      service:
        name: nginx
        state: started
        enabled: yes
```

- Playbook start
- Name of the play
- Target hosts
- Run as sudo
- List of tasks
- First task
- Module: yum
- Module arguments
- Second task
- Module: service
- Module arguments

9. YAML Best Practices

- Use 2 spaces for indentation.
- Be consistent with indentation.
- Use meaningful key names.
- Add comments for better understanding.
- Validate YAML using online tools or linters.

Some Useful Tools

- ★ YAML Lint
- ★ Online YAML Parser
- ★ VS Code (YAML Extension)
- ★ ansible-lint (for Ansible)

Summary

YAML is simple, powerful and human friendly. It is widely used in DevOps tools like Ansible, Kubernetes, Docker Compose, Jenkins pipelines etc. Understanding YAML is essential for automation and configuration management.





ANSIBLE

ANSIBLE PLAYBOOK

Automation Made Simple !! 😊

Playbooks are written in YAML format. They define a set of tasks to be executed on remote hosts in a desired order. 😊

1. About Playbooks

- Playbooks are YAML files that define the configuration, deployment, and management of systems.
- They are human-readable, version-controlled and reusable.
- Playbooks contain one or more plays.
- Each play targets a group of hosts and runs a list of tasks.
- Playbooks are the heart of Ansible automation.

2. Playbook Language Example - YAML

```
---
- name: Install and start Nginx      # Play name
  hosts: webservers                 # Target hosts or group
  become: yes                       # Privilege escalation
  vars:
    pkg_name: nginx
    svc_name: nginx
  tasks:
    - name: Install Nginx package
      ansible.builtin.package:
        name: "{{ pkg_name }}"
        state: present
    - name: Start and enable Nginx service
      ansible.builtin.service:
        name: "{{ svc_name }}"
        state: started
        enabled: yes
    - name: Copy custom index page
      ansible.builtin.copy:
        src: index.html
        dest: /usr/share/nginx/html/index.html
        notify: restart nginx
  handlers:
    - name: restart nginx
      ansible.builtin.service:
        name: "{{ svc_name }}"
        state: restarted
```

3. How to Write Playbooks

Steps:

1. Define the play (hosts, become, vars, etc.)
2. Add tasks to perform actions.
3. (Optional) Add handlers, vars, files, templates etc.
4. Run the playbook using: `ansible-playbook <playbook.yml>`

4. Tasks in Playbooks

- Tasks are the individual actions executed on the target hosts.
- Each task uses a module to perform a specific operation.
- Tasks run in the order they are defined.

5. Understanding about various tasks in playbook

Task / Module	Purpose	Example
package / yum / apt	Install or remove packages	- package: name=nginx state=present
service	Manage services	- service: name=nginx state=started
copy	Copy files to remote host	- copy: src=file dest=/tmp/file
template	Render and copy Jinja2 templates	- template: src=conf.j2 dest=/etc/conf.conf
file	Manage files and directories	- file: path=/opt/test state=directory
user	Manage user accounts	- user: name=devops state=present
command	Run shell commands	- command: /usr/bin/uptime
shell	Run shell commands through shell	- shell: ls -l /tmp
lineinfile	Ensure a line in a file	- lineinfile: path=/etc/hosts line='192.168.1.10 web1'
debug	Print messages for debugging	- debug: msg="Hello Ansible"
uri	Interact with web services	- uri: url=http://example.com

6. Introduction to Handlers and Variables

- Variables store values that can be reused in tasks.
- Handlers run tasks only when notified by other tasks.

Variables

- Store dynamic values
- Defined in playbook, vars section, or external files
- Used with `{{ variable_name }}`

Handlers

- Used for tasks like restarting services
- Executed only when notified
- Defined under handlers section

7. Using Handlers and Variables in Playbook

Example Playbook:

```
---
- name: Configure Nginx with Handler and Variables
  hosts: webservers
  become: yes
  vars:
    nginx_port: 8080
    nginx_conf_src: nginx.conf.j2
  tasks:
    - name: Install Nginx
      package:
        name: nginx
        state: present
    - name: Deploy Nginx config
      template:
        src: "{{ nginx_conf_src }}"
        dest: /etc/nginx/nginx.conf
        notify: restart nginx
    - name: Ensure Nginx is running
      service:
        name: nginx
        state: started
        enabled: yes
  handlers:
    - name: restart nginx
      service:
        name: nginx
        state: restarted
```

How it works:

1. Template task changes the config file.
2. It notifies the handler "restart nginx".
3. Handler runs only if notified.

Variables can also be defined separately:

```
# vars.yml
nginx_port: 8080
nginx_conf_src: nginx.conf.j2
```

Use in playbook:

```
vars_files:
  - vars.yml
```

8. Become (Privilege Escalation)

- Many tasks require elevated privileges.
- Ansible provides `become` to run tasks as another user (usually root).
- Default `become_method` is `sudo`.

Ways to use become

```
become: yes
become_user: root      # Run tasks as root

become: yes
become_user: postgres  # Run tasks as postgres user

become_method: su      # Use su instead of sudo

become_flags: "-i"     # Additional flags for become
```

Example:

```
---
- name: Create user and deploy app
  hosts: appservers
  become: yes
  tasks:
    - name: Create application user
      user:
        name: appuser
        state: present
    - name: Copy application files
      copy:
        src: app/
        dest: /opt/myapp/
        owner: appuser
        group: appuser
        mode: '0755'
    - name: Start application service
      service:
        name: myapp
        state: started
        enabled: yes
```

Notes

- ✓ `become` increases security and control.
- ✓ Always use least privilege principle.
- ✓ You can also set privilege escalation in `ansible.cfg`

Summary

- ✓ Playbooks are YAML files that define automation.
- ✓ Tasks use modules to perform operations.
- ✓ Variables make playbooks dynamic and reusable.
- ✓ Plays contain tasks, variables and handlers.
- ✓ Handlers run tasks only when notified.
- ✓ `Become` helps to run tasks with elevated privileges.





ANSIBLE

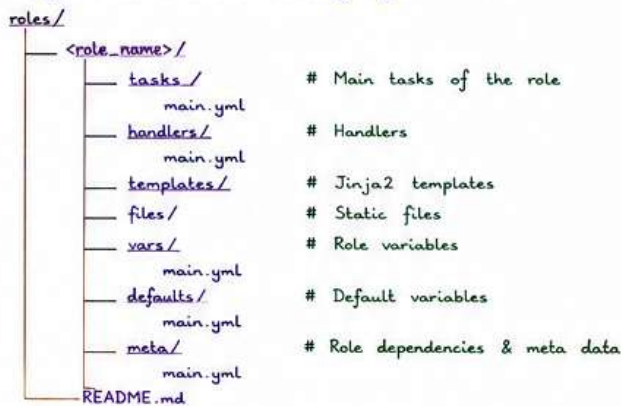
ANSIBLE - ROLES

Roles help to organize playbooks and tasks in a reusable and structured way. They promote reusability, maintainability and sharing.

Roles are a way to automatically load variables, tasks, files and handlers based on a standard directory structure. !!

1. Role Directory Structure

- Every role has a standard directory layout.



Notes :

- ✓ tasks/main.yml is the entry point for tasks.
- ✓ defaults/ has lowest priority, vars/ has higher priority.

2. Using Roles

- Roles are used in a playbook under the roles section.

Example Playbook :

```

- name: Install and configure Nginx
  hosts: webservers
  become: yes
  roles:
    - common
    - nginx
    - firewall

```

Explanation :

- Ansible looks for roles in roles/ directory relative to the playbook.
- Roles are executed in the order listed.

3. Role Duplication and Execution

- A role can be used multiple times with different variables.
- You can also run a role directly from command line.

Example - Use same role twice with different vars :

```

- hosts: webservers
  become: yes
  roles:
    - role: nginx
      vars:
        nginx_port: 80
    - role: nginx
      vars:
        nginx_port: 8080

```

Run role directly :

```

ansible-playbook -i inventory
site.yml --tags nginx

or

ansible-playbook -i inventory
roles/nginx/tasks/main.yml

```

4. Role Default Variables

- Default variables are defined in defaults/main.yml.
- They have the lowest precedence and can be overridden.

Example defaults/main.yml :

```

---
nginx_port: 80
nginx_user: www-data

```

Override in playbook/vars :

```

- hosts: webservers
  vars:
    nginx_port: 8080

```

5. Role Dependencies

- Roles can depend on other roles.
- Define in meta/main.yml

Example meta/main.yml :

```

---
dependencies:
  - role: common
  - role: firewall

```

Explanation :

- Dependencies are executed before the current role.
- Useful to avoid repeating code.

6. Role Search Path

- Ansible searches for roles in the following paths (in order) :

1. roles/ directory in the playbook directory
2. ROLES_PATH environment variable (colon-separated paths)
3. /etc/ansible/roles/
4. ~/.ansible/roles/

Example :

Set custom role path

export ANSIBLE_ROLES_PATH=/opt/ansible/roles:/usr/share/ansible/roles

7. Ansible Galaxy

- Ansible Galaxy is a central hub for sharing roles.
- You can search, download and publish roles.

Common Commands :

ansible-galaxy search <keyword>	Search roles
ansible-galaxy install <role_name>	Install a role
ansible-galaxy install <namespace.role>	Install from namespace
ansible-galaxy list	List installed roles

Example :

```

ansible-galaxy install geerlingguy.nginx
# Installs into ~/.ansible/roles/

```

8. INCLUDING AND IMPORTING

8. Includes vs. Imports

Includes	Imports
Processed at runtime	Processed at playbook parse time
Can be conditional (with when)	Cannot be conditional
Can include dynamic files	Static - files must exist at parse time
More flexible	Faster execution

9. Importing Playbooks

- Use import_playbook to include another playbook.

Example :

```

# site.yml
- import_playbook: common.yml
- import_playbook: webservers.yml

```

Note :

The imported playbook is parsed before execution.

10. Including and Importing Task Files

Include Tasks (runtime)

```

- tasks:
  - include_tasks: install.yml
    when: ansible_os_family == "RedHat"

# install.yml
- name: Install httpd
  yum:
    name: httpd
    state: present

```

Import Tasks (parse time)

```

- tasks:
  - import_tasks: install.yml

# install.yml
- name: Install httpd
  yum:
    name: httpd
    state: present

```

11. Including and Importing Roles

Include Role (runtime)

```

- hosts: webservers
  roles:
    - role: nginx
      when: install_nginx | bool

```

Import Role (parse time)

```

- hosts: webservers
  roles:
    - import_role:
        name: nginx

```

Key Takeaways

- ✓ Roles organize tasks and make playbooks reusable.
- ✓ Use roles/ directory and follow standard structure.
- ✓ Defaults have low precedence; vars override them.
- ✓ Use meta/main.yml for dependencies.
- ✓ Ansible Galaxy helps to find and share roles.
- ✓ Includes are dynamic; Imports are static.
- ✓ Use includes/imports for tasks, roles and playbooks wisely.



MONITORING AND OBSERVABILITY

Logging and Metrics : Prometheus, Grafana, and ELK stack
for application tracking and system alerts.

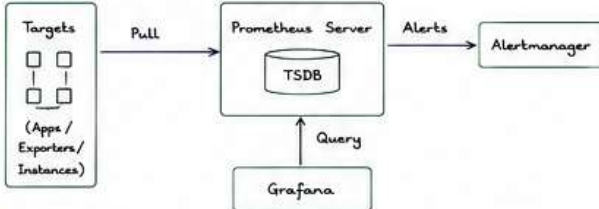
1. LOGGING AND METRICS OVERVIEW

- Monitoring tells us WHAT is happening.
- Observability helps us understand WHY it is happening.
- Metrics = Numeric data over time (CPU, memory, requests)
- Logs = Discrete events (errors, access, traces)
- Traces = Request flow across services (Jaeger, Zipkin)

2. PROMETHEUS (Metrics Collection)

- Open-source monitoring and alerting toolkit.
- Pull-based model: Prometheus scrapes metrics from targets.
- Stores data in time-series database.
- Uses PromQL for querying.

PROMETHEUS ARCHITECTURE



Key Components

- Prometheus Server : collects & stores metrics
- Exporters : expose metrics (node-exporter, app-exporter)
- Alertmanager : handles alerts and notifications
- Grafana : visualization and dashboards

3. GRAFANA (Visualization)

- Open-source analytics & visualization platform.
- Connects to multiple data sources (Prometheus, InfluxDB, MySQL)
- Build dashboards with panels, alerts, annotations.
- Share dashboards and set user permissions.



Features:

- ✓ Rich dashboards
- ✓ Alerts & notifications (Email, Slack, Telegram, Webhook)
- ✓ Supports variables and templating
- ✓ Wide plugin ecosystem

4. ELK STACK (Log Management)

ELK = Elasticsearch + Logstash + Kibana

- Centralized logging and log analytics.
- Helps in tracking application logs and generating alerts.



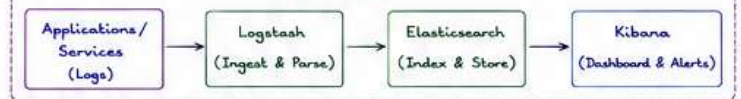
Components:

- Logstash : Collects logs from multiple sources, parses & filters.
- Elasticsearch : Stores, indexes and makes logs searchable.
- Kibana : Visualizes logs through dashboards and graphs.

Common Use Cases:

- ✓ Application error tracking
- ✓ User activity monitoring
- ✓ Security event analysis
- ✓ System audit and compliance

ELK LOG FLOW



5. SYSTEM ALERTS

- Alerts notify teams about issues before users are impacted.
- Alertmanager (in Prometheus) handles alert routing.
- Can integrate with Email, Slack, PagerDuty, Webhooks, etc.

Example Alert Rule (Prometheus Alert)

```
groups:
- name: high-cpu-alerts
  rules:
  - alert: HighCPUUsage
    expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "High CPU usage detected on {{ $labels.instance }}"
      description: "CPU usage is above 80% for more than 5 minutes."
```



DEVSECOPS

Vulnerability scanning, secrets management, and shift-left security implementation.



1. VULNERABILITY SCANNING

- Identify security flaws early in the SDLC.
- Scan code, dependencies, containers, infrastructure & images.
- Helps fix vulnerabilities before production.

Type	Tools	What it scans	Example
SAST	SonarQube, Checkmarx	Source Code	SQL Injection, XSS
SCA	OWASP Dependency-Check, NPM Audit, Snyk	Dependencies	Outdated / Vulnerable Libraries
DAST	OWASP ZAP, Burp Suite	Running Application	SQLi, CSRF
Container Scan	Trivy, Clair, Anchore	Docker Images	CVE, Misconfigurations
IaC Scan	Checkov, tfsec	Terraform, CloudFormation, Kubernetes Manifests	Misconfigurations

Best Practices

- ✓ Run scans in CI pipeline
- ✓ Fail build on critical issues
- ✓ Generate reports & track in dashboard
- ✓ Rescan after fixes

CI PIPELINE WITH SCAN



2. SECRETS MANAGEMENT

- Store and manage sensitive data securely (passwords, keys, tokens, certs).
- Never hardcode secrets in code or configs.
- Use dedicated secrets management tools.

Tools	Features
HashiCorp Vault	Centralized secrets, dynamic secrets, audit logs
AWS Secrets Manager	Managed service for storing and rotating secrets
Azure Key Vault	Store keys, secrets, certs in Azure
Google Secret Manager	Secure storage with IAM integration

Example (Vault)

```
# Store a secret
vault kv put secret/db password="MyPass@123"

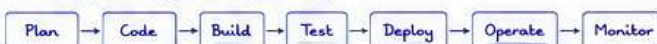
# Read a secret
vault kv get secret/db
```

Best Practices

- ✓ Store secrets outside code
- ✓ Rotate secrets regularly
- ✓ Use Least Privilege Access
- ✓ Enable audit logging

3. SHIFT-LEFT SECURITY

- Integrate security early in the development lifecycle.
- Fix issues early → Cost effective, faster, safer.
- Security is everyone's responsibility.



Security Activities (Shift-Left)

Threat Modeling	Secure Coding	SCA	DAST	Runtime Security
Security Requirements	SAST	Image Scan	Pen Testing	Log Monitoring
	Secrets Scan	IaC Scan	Fuzz Testing	Vulnerability Mgmt.

4. DEVSECOPS BENEFITS

- ✓ Early detection of vulnerabilities
- ✓ Reduced security risks
- ✓ Compliance and governance
- ✓ Faster feedback & secure releases
- ✓ Automation and consistency

Key Takeaways

- Monitor everything (metrics, logs, traces)
- Use right tools: Prometheus, Grafana, ELK
- Alert on issues before impact
- Secure early, secure always
- Automate security in every pipeline

OBSERVABILITY + SECURITY = RELIABLE & SECURE SYSTEMS

Monitor to understand. Secure to protect. Automate to scale. 😊